

ZIGChain

DualDefense Audit

Performed through HackenProof

FINDING

Phantom LP supply causes permanently locked pool reserves

HIGH IMPACT

HIGH LIKELIHOOD

AUTHOR

Nathan Byarugaba

DATE

December 21, 2025

TARGET

x/dex (Cosmos SDK)

Executive Summary

This report documents a high-severity finding identified during a security review of the ZIGChain codebase, conducted via the HackenProof platform. The vulnerability affects the x/dex module's liquidity pool accounting and results in the permanent loss of a portion of every affected pool's reserves.

Finding at a Glance

FINDING	Phantom LP supply causes permanently locked pool reserves
SEVERITY	Critical (High Impact × High Likelihood)
MODULE	x/dex (Cosmos SDK)
REPOSITORY	github.com/hackenproof-public/zigchain
CLASS	Accounting / Funds at Risk
STATUS	Reproduced via on-chain PoC test

Severity Rating

HIGH IMPACT

HIGH LIKELIHOOD

PERMISSIONLESS

FUNDS LOCKED

Key Takeaway

The x/dex keeper tracks more LP supply in pool state than it actually mints. When all real LP tokens are burned, a fraction of every affected pool's reserves remains permanently unwithdrawable — funds are lost at the protocol level with no permissionless path to recovery.

1. Finding Details

1.1 Title

Phantom LP supply causes permanently locked pool reserves

1.2 Target

REPOSITORY	github.com/hackenproof-public/zigchain
MODULE	x/dex/keeper
PLATFORM	HackenProof

1.3 Affected Source Locations

Description	File	Lines
Core mint / accounting on pool creation	x/dex/keeper/msg_server_create_pool.go	L231-L303
Minimal liquidity lock vs userShares computation	x/dex/keeper/msg_server_create_pool.go	L318-L346
Pool LP supply decremented by burned amount on removal	x/dex/keeper/msg_server_remove_liquidity.go	L14-L115
Withdraw proration uses pool.LpToken.Amount	x/dex/keeper/msg_server_remove_liquidity.go	L125-L157

2. Summary

The x/dex module implements a “minimal liquidity lock” by tracking a larger LP total supply in pool state than the LP tokens actually minted. This creates **phantom LP supply** and makes a portion of every affected pool's reserves permanently unwithdrawable, even after all real LP tokens have been burned.

3. Finding Description

During pool creation, the keeper calculates:

```
lpAmount = floor(sqrt(base * quote)) // stored in pool.LpToken.Amount
userShares = lpAmount - MinimalLiquidityLock // minted and sent to user
```

However, only userShares is actually minted, while the pool state stores the larger lpAmount as the total LP supply. Later, on liquidity removal, the system computes withdrawals as:

```
coinsOut = poolCoins * (lptoken.Amount / pool.LpToken.Amount)
```

Because pool.LpToken.Amount includes the lock amount that was never minted to anyone, the system behaves as if there exists a permanently locked LP position — without ever issuing locked LP

tokens. As a direct consequence, a fraction of pool reserves will remain stuck forever and no actor on chain can ever claim them.

4. Impact Explanation

HIGH IMPACT

- **Permanent freezing of funds.** Even after every LP token that any user can ever own is burned, the pool retains a positive reserve balance with no entity authorised to claim it.
- **Affects every pool with a non-zero minimum liquidity lock.** Any pool created while `MinimalLiquidityLock > 0` is vulnerable; the stuck fraction can be significant for small pools near the lock threshold.
- **Protocol-level loss of funds.** The defect is in core accounting, not in a peripheral feature, and there is no permissionless recovery path. This satisfies the *High Impact* criterion of irrecoverable user funds.

5. Likelihood Explanation

HIGH LIKELIHOOD

- **Permissionless trigger.** Any user can create a pool — no governance, whitelist, or special role is required.
- **No resource barrier.** No special token balances are needed beyond the minimum liquidity that pool creation already requires.
- **Deterministic outcome.** The bug is not probabilistic — it follows directly and reliably from the minting and accounting logic on every invocation.

Because the issue is trivially triggerable without privileges or meaningful resources, and produces the loss outcome deterministically, it meets the *High Likelihood* criterion.

6. Recommendation

Fix Summary

Make the minimum liquidity lock **real** instead of phantom: ensure that the LP supply tracked in pool state matches the LP tokens actually minted, and represent the “locked” portion as real LP tokens sent to an address that cannot sign (e.g. the pool account address).

6.1 Suggested Pattern

A safe pattern is:

- Mint the full `lpAmount` as total LP supply.
- Send `userShares` to the depositor.
- Send `minimalLock` LP tokens to an unspendable sink (e.g. the pool account address).
- Keep `pool.LpToken` tracking `lpAmount` — which now equals real minted supply.

6.2 Illustrative Patch

Conceptual Go snippet:

```
// After computing lpAmount and userShares:
// minimalLock = lpAmount - userShares

// 1) Mint total LP = lpAmount
_ = k.bankKeeper.MintCoins(ctx, types.ModuleName, sdk.NewCoins(lpAmount))

// 2) Send userShares to receiver
_ = k.bankKeeper.SendCoinsFromModuleToAccount(
    ctx, types.ModuleName, receiver, sdk.NewCoins(userShares))

// 3) Send minimalLock LP to an unspendable sink (e.g. pool address)
minimalLockCoin := sdk.NewCoin(
    poolIDString, lpAmount.Amount.Sub(userShares.Amount))
poolAddr := types.GetPoolAddress(poolIDString)
_ = k.bankKeeper.SendCoinsFromModuleToAccount(
    ctx, types.ModuleName, poolAddr, sdk.NewCoins(minimalLockCoin))

// 4) Ensure pool.LpToken tracks lpAmount
// (which now matches real minted supply)
pool.LpToken = lpAmount
```

This preserves the intended minimum liquidity lock behaviour without introducing phantom LP supply.

7. Validation Steps

7.1 Proof of Concept

A reproducible Proof-of-Concept test was added at:

```
x/dex/keeper/critical_poc_phantom_lp_locks_funds_test.go
```

The test creates a pool with `MinimalLiquidityLock = 1000`, deposits base and quote, then removes all minted LP. It asserts that the LP supply on chain becomes zero while the pool reserves remain positive — proving the reserves are permanently locked.

7.2 Running the Test

```
go test ./x/dex/keeper \
  -run TestCriticalPoC_PhantomLPSupply_LocksPoolReserves \
  -count=1
```

7.3 Full PoC Source

The complete Go test source for the PoC is reproduced on the following pages.

Appendix A — PoC Test Source

x/dex/keeper/critical_poc_phantom_lp_locks_funds_test.go

```
package keeper_test

import (
    "testing"

    sdk "github.com/cosmos/cosmos-sdk/types"
    minttypes "github.com/cosmos/cosmos-sdk/x/mint/types"
    "github.com/stretchr/testify/require"

    keepertest "zigchain/testutil/keeper"
    "zigchain/testutil/sample"
    "zigchain/x/dex/keeper"
    "zigchain/x/dex/types"
)

// PoC: demonstrate that with MinimalLiquidityLock > 0,
// pool accounting can create "phantom LP supply" where
// total LP tracked in state exceeds actual minted supply.
// After burning all LP tokens users can possibly own, the
// pool still retains positive reserves that are permanently
// unwithdrawable (no permissionless recovery).
func TestCriticalPoC_PhantomLPsupply_LocksPoolReserves(t *testing.T) {
    creator := sample.AccAddress()
    signer := sdk.MustAccAddressFromBech32(creator)

    k, ctx, bankKeeper := keepertest.DexKeeperWithBank(t, nil)
    srv := keeper.NewMsgServerImpl(k)

    // Configure params to isolate the issue:
    // - No creation fee
    // - MinimalLiquidityLock > 0 (the feature that triggers the bug)
    params := types.DefaultParams()
    params.CreationFee = 0
    params.MinimalLiquidityLock = 1000
    require.NoError(t, k.SetParams(ctx, params))
}
```

```
// Choose amounts so that:
// lpAmount = sqrt(base*quote) = 1002 (perfect square)
// userShares = lpAmount - minimalLock = 2
base := sample.Coin("base", 1002)
quote := sample.Coin("quote", 1002)

// Fund creator with base+quote
funding := sdk.NewCoins(base, quote)
require.NoError(t, bankKeeper.MintCoins(
    ctx, minttypes.ModuleName, funding))
require.NoError(t, bankKeeper.SendCoinsFromModuleToAccount(
    ctx, minttypes.ModuleName, signer, funding))

// Create pool
createRes, err := srv.CreatePool(ctx, &types.MsgCreatePool{
    Creator: creator,
    Base: base,
    Quote: quote,
})
require.NoError(t, err)
require.NotNil(t, createRes)

poolID := createRes.PoolId
require.Equal(t, uint64(2), createRes.LpToken.Amount.Uint64(),
    "creator should only receive userShares=2 LP tokens")

// Confirm on-chain total LP supply == minted amount (2),
// not the pool state's total (1002).
supply := bankKeeper.GetSupply(ctx, poolID)
require.Equal(t, uint64(2), supply.Amount.Uint64(),
    "only userShares should be minted on create")

pool, found := k.GetPool(ctx, poolID)
require.True(t, found)
require.Equal(t, uint64(1002), pool.LpToken.Amount.Uint64(),
    "pool state tracks total LP = lpAmount (incl. minimal lock)")

// Remove ALL LP tokens the user can possibly own
_, err = srv.RemoveLiquidity(ctx, &types.MsgRemoveLiquidity{
    Creator: creator,
    Lptoken: createRes.LpToken,
})
require.NoError(t, err)

// After burning the only minted LP token,
// total supply becomes zero...
supplyAfter := bankKeeper.GetSupply(ctx, poolID)
require.True(t, supplyAfter.Amount.IsZero(),
    "all minted LP should be burned")
```



```
// ...but the pool still retains positive reserves,  
// proving funds are stuck with no LP token claims.  
poolAddr := types.GetPoolAddress(poolID)  
baseLeft := bankKeeper.GetBalance(ctx, poolAddr, base.Denom)  
quoteLeft := bankKeeper.GetBalance(ctx, poolAddr, quote.Denom)  
  
require.True(t, baseLeft.Amount.IsPositive(),  
    "pool should still have leftover base reserves (locked)")  
require.True(t, quoteLeft.Amount.IsPositive(),  
    "pool should still have leftover quote reserves (locked)")  
  
// And pool state still tracks non-zero LP total  
// despite supply being zero.  
poolAfter, found := k.GetPool(ctx, poolID)  
require.True(t, found)  
require.Equal(t, uint64(1000), poolAfter.LpToken.Amount.Uint64(),  
    "pool LP total decreases by burned amount but remains "+  
    "non-zero (phantom lock)")  
}
```

End of report — Submitted via HackenProof by Nathan Byarugaba, 21 December 2025.